German University in Cairo

Goalkeeper Robotic Arm

Image Processing Project

Team Number: 17

Menna Walaa	55-0908
Eman Elezaby	55-5538
Farah Elzeky	55-7554
Khadija Sherif	55-2841
Nabil Mohamed	55-1543
Seif Alhennawy	55-18172

May 5, 2026

1 Literature Review

1.1 Introduction

Vision-based robotic systems play a significant role in applications requiring rapid decision-making and precise actuation. Robotic soccer and ball interception tasks present particular challenges due to the high velocity of the object and limited reaction time. A goalkeeper robotic arm with one degree of freedom (1DoF) depends on accurate visual detection and reliable motion prediction to determine the rotational movement necessary to block an incoming ball. The effectiveness of such systems relies on the integration of image processing techniques with embedded motor control hardware.

1.2 Goalkeeper Robotics and Ball Interception

Recent research demonstrates the feasibility of robotic ball interception using visual feedback. Harikumar et al. [1] proposed a colored ball tracking approach for humanoid goalkeeper robots, emphasizing fast image processing for dynamic environments. Santos et al. [2] developed a vision-based robotic catching system that combined trajectory prediction with real-time actuation to enhance interception accuracy.

Similarly, Patel et al. [3] investigated robotic arm mechanisms for blocking fast-moving balls, confirming that low-degree-of-freedom systems can perform effectively when supported by accurate trajectory estimation. These studies highlight that reliable detection and predictive control are fundamental to goalkeeper robotic performance.

1.3 Image Processing Techniques for Ball Detection

Classical image processing techniques remain widely used in real-time embedded applications due to their computational efficiency. HSV color space segmentation is frequently employed to isolate a colored ball from the background [4]. By selecting appropriate threshold ranges, the ball region can be distinguished even under moderate illumination variations.

To improve segmentation quality, morphological operations such as erosion and dilation are applied to eliminate noise and refine object boundaries. After segmentation, contour detection or the Hough Circle Transform can be used to determine the center coordinates of the ball [5]. These techniques provide geometric features necessary for tracking while maintaining low processing latency.

1.4 Tracking and Trajectory Prediction

Detection alone is insufficient for interception; motion estimation is also required. Tracking algorithms estimate the ball position across consecutive frames to compute velocity

and predict future positions. The Kalman filter is commonly applied in robotic vision systems to smooth measurements and model motion dynamics [6].

For short-distance goalkeeper applications, linear motion models are generally adequate to approximate the ball's trajectory [2]. The predicted interception point can then be translated into an angular displacement command for the 1DoF robotic arm. Such predictive filtering significantly improves response timing and blocking accuracy.

1.5 Hardware and Electronics Implementation

The successful implementation of a goalkeeper robotic arm depends on efficient integration between vision processing and motor actuation. Embedded platforms such as Raspberry Pi can execute OpenCV-based algorithms including filtering, segmentation, and tracking [4]. These platforms offer sufficient computational capability for real-time performance in compact systems.

Servo motors are typically used for arm rotation due to their precise angular positioning. Arduino microcontrollers generate Pulse Width Modulation (PWM) signals to control servo movement. In many implementations, Proportional–Integral–Derivative (PID) controllers are employed to reduce steady-state error and enhance stability [7]. The coordination between image processing output and motor control circuitry enables fast and accurate response to predicted ball positions.

1.6 Deep Learning-Based Detection

Deep learning approaches, particularly Convolutional Neural Networks (CNNs), have recently been explored for object detection tasks including ball tracking [8]. These methods improve robustness under complex backgrounds and lighting conditions. However, their computational requirements are higher than classical techniques, which may limit their suitability for lightweight embedded systems. Consequently, traditional image processing methods remain practical for low-cost 1DoF goalkeeper implementations.

2 Proposed Project Plan

The proposed goalkeeper system follows a sequence of image processing and control steps to detect a moving ball and move a robotic goalkeeper to block it.

1. **Image Acquisition**
   A camera connected to the Raspberry Pi captures continuous frames of the goal area where the ball movement occurs.

2. **Image Pre-processing**
   The captured frames are processed to improve image quality. Image filtration techniques such as Gaussian filtering are applied to reduce noise, and the image may be converted from RGB to HSV or grayscale to simplify further processing.

3. **Ball Segmentation**
   The ball is separated from the background using segmentation techniques. Color thresholding is applied to isolate the ball based on its color, followed by morphological operations such as erosion and dilation to remove noise and refine the detected region.

4. **Ball Detection and Localization**
   After segmentation, contour detection is used to identify the ball. The centroid of the detected contour is calculated to determine the ball position within the image frame.

5. **Motion Analysis and Decision Making**
   Using the detected ball positions across consecutive frames, the system estimates the ball movement and determines the appropriate position for the goalkeeper to intercept the ball.

6. **Actuation**
   The Raspberry Pi sends control signals to an Arduino microcontroller, which drives the motors or servos responsible for moving the robotic goalkeeper to block the ball.

3 List of Hardware Components

Component	**Quantity**	**Price (EGP)**
Raspberry Pi 5 (4GB)	1	5950
Raspberry Pi Camera Module V2 (8MP IMX219)	1	800
Arduino UNO Rev3 (Clone)	1	450
FS5103R 360° Continuous Servo Motor	1	350
Acrylic Case for Raspberry Pi Camera	1	95

Table 1: Hardware components required for the system

4 System Proposed DiagramS

4.1 Flow of Data Diagram

The system block diagram illustrates the architecture of the goalkeeper robotic arm. A camera captures image frames of the incoming ball and sends them to the Raspberry Pi, where image processing and color-based ball tracking are performed. Based on the detected ball position, the Raspberry Pi generates control commands and transmits them to the Arduino microcontroller via serial communication. The Arduino outputs PWM and digital signals to the motor driver, which actuates the robotic arm motors to block the ball. A regulated power supply provides the required logic and motor voltages to all system components.

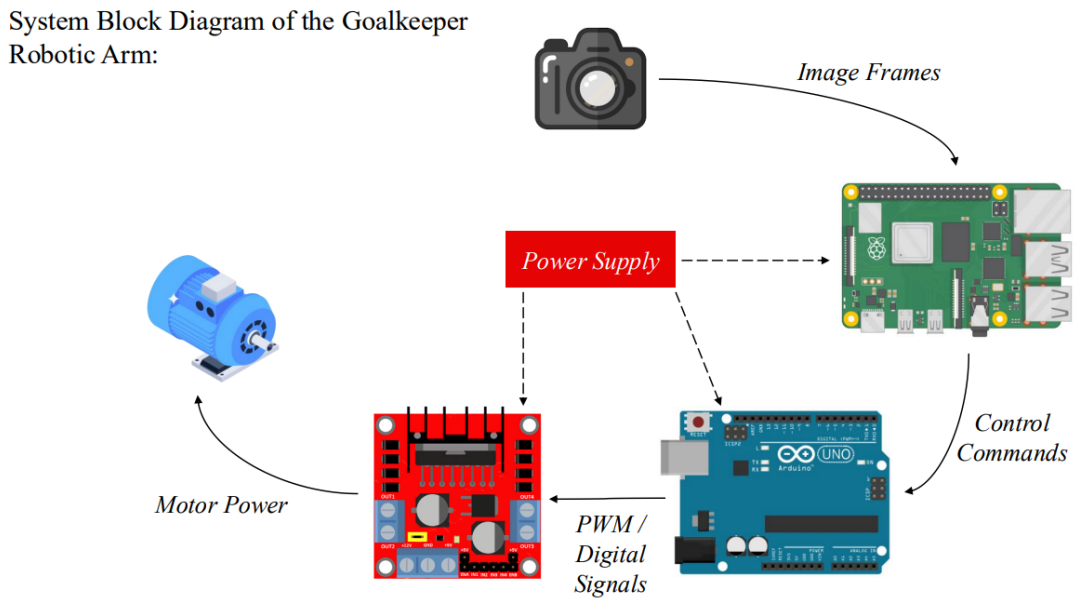

Figure 1: Schematic Diagram

4.2 Circuit Design

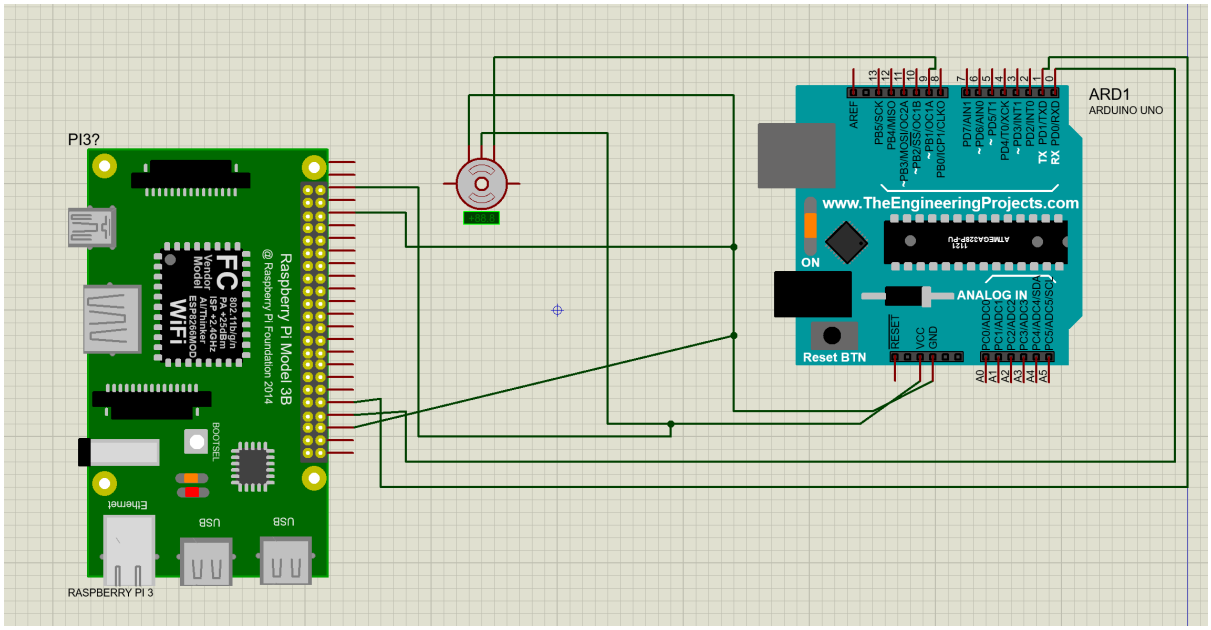

Figure 2: Circuit Design

4.3 Hardware Design

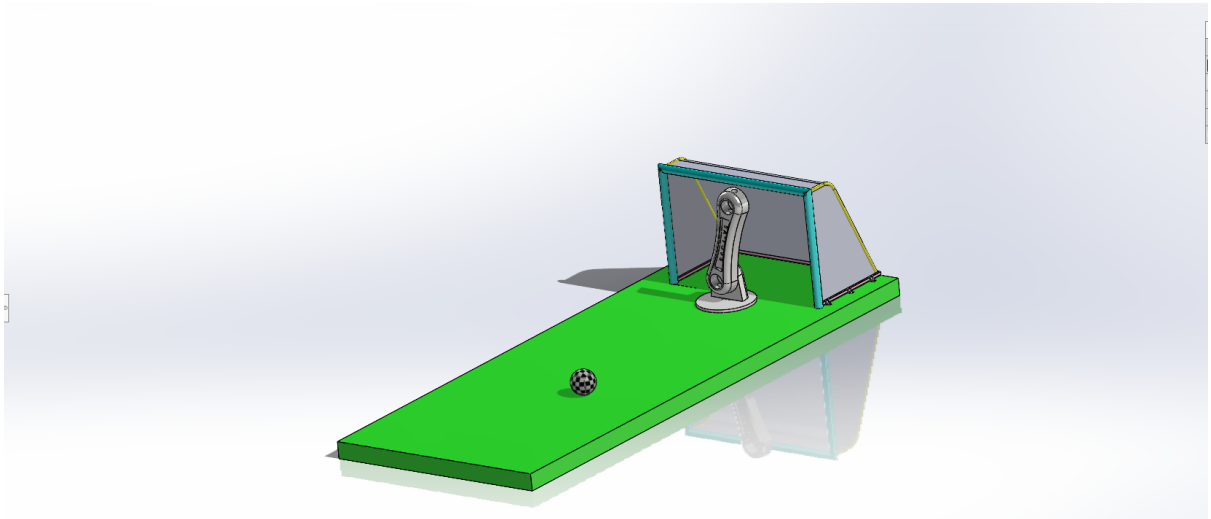

Figure 3: Solidworks Design

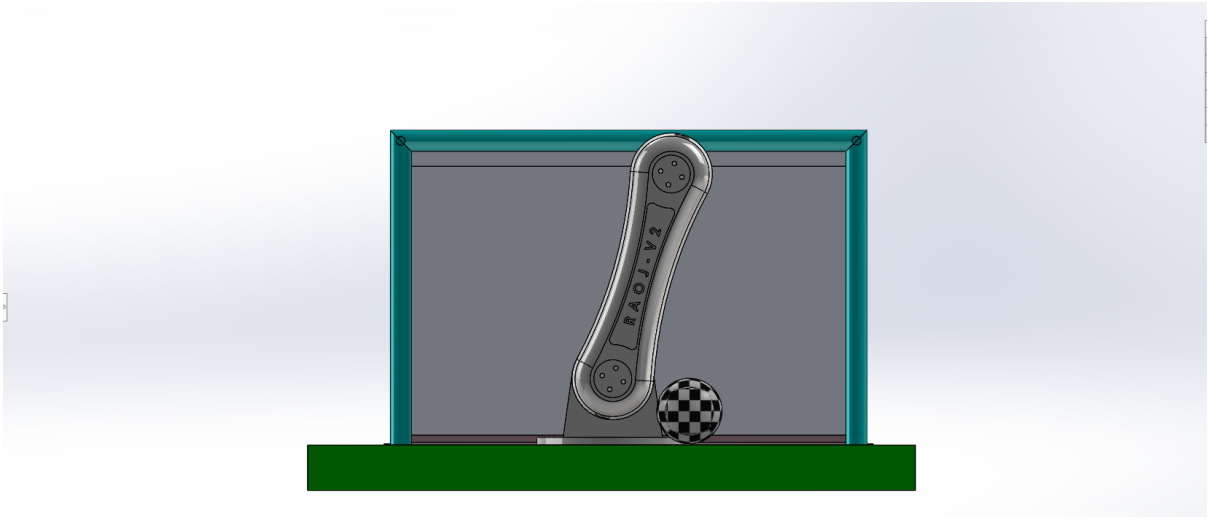

Figure 4: Solidworks Design

5 Methodology

5.1 Overall Hardware Fabrication and Connections

The complete system was physically assembled by integrating the Raspberry Pi, Arduino Uno, camera module, and servo motor into a unified setup.

The Raspberry Pi is connected to the camera module using the CSI interface, enabling real-time image acquisition. The Raspberry Pi communicates with the Arduino Uno through a USB serial connection, where control commands are transmitted based on image processing results.

The Arduino is responsible for generating PWM signals to drive the servo motor. The servo motor performs rotational motion corresponding to the received commands, allowing the robotic arm to move toward the detected object position.

5.2 Initial Approach: Brightness-Based Control

In the initial stage of the project, brightness was used as the primary image feature to control the motor. The captured frame was converted to grayscale, and the average pixel intensity was calculated.

Based on the brightness value:

- High brightness resulted in clockwise motor rotation
- Low brightness resulted in counter-clockwise motor rotation

This method was simple and computationally efficient. However, it showed several limitations. The system was highly sensitive to lighting conditions, and small variations in illumination caused unstable motor behavior. Additionally, this approach did not provide accurate positional information about the object within the frame.

5.3 Improved Approach: Binary Image Processing and Grid-Based Control

To overcome the limitations of the initial approach, a new method based on binary image processing and grid-based positioning was implemented.

Each captured frame is converted into a binary image where the ball appears as a white region and the background is black. This simplifies the detection process and reduces computational complexity.

After segmentation, contour detection is applied to identify the object. The largest contour is selected, and its centroid is calculated to determine the position of the ball within the frame.

To control the motor, the frame is divided into a 3×3 grid. Each grid cell corresponds to a predefined servo angle, as illustrated in the next figure.

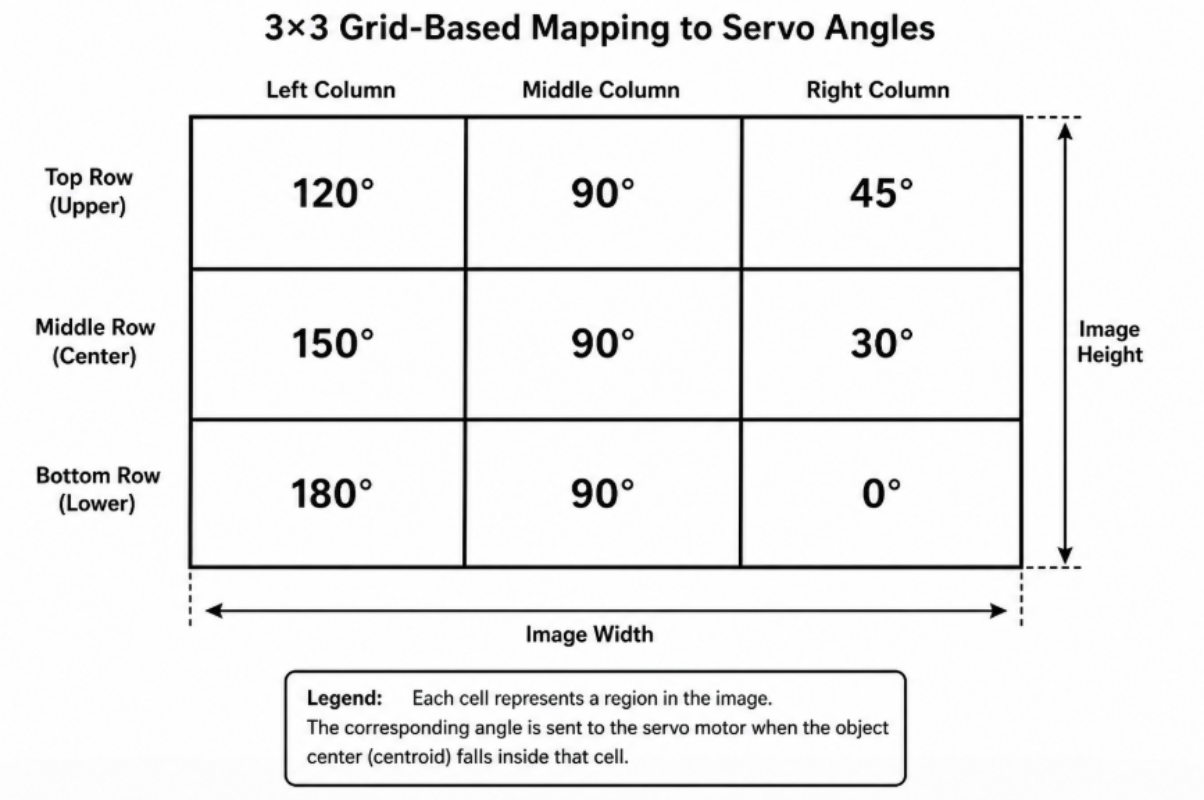

Figure 5: Mapping between image grid regions and corresponding servo angles used for discrete motor control

When the object is detected in a specific cell, the corresponding angle is directly sent to the Arduino. This discrete mapping reduces noise sensitivity, eliminates unnecessary small movements, and improves overall system stability.

5.4 Final System Operation

The final system operates in a continuous closed-loop process:

1. Capture image frame using the camera
2. Convert the frame into a binary image
3. Detect the object using contour detection
4. Determine the object's grid position
5. Select the corresponding servo angle
6. Send the angle command to the Arduino
7. Move the servo motor accordingly

This process ensures real-time tracking of the object and immediate actuator response.

5.5 Performance Considerations

The system was designed to achieve real-time performance while maintaining simplicity in implementation. The use of binary segmentation reduces processing time, making it suitable for execution on the Raspberry Pi.

The grid-based control approach further improves performance by avoiding continuous angle computation and reducing actuator jitter. This results in a more stable and responsive system compared to the initial brightness-based method.

Additionally, the use of embedded hardware provides a compact and cost-effective solution for real-time robotic applications.

6 Image Processing and Control Implementation

6.1 Overview of the Implemented System

The implemented system integrates real-time image processing with embedded motor control. The Raspberry Pi captures live video frames using the camera module and processes these frames to detect the target object (white ball). Based on the detected position of the object, control commands are generated and transmitted to the Arduino to actuate the servo motor.

This implementation forms a closed-loop control system where visual feedback directly influences actuator response.

6.2 Image Processing Steps

The following steps are performed on each captured frame:

1. **Frame Acquisition**
   The camera continuously captures frames at a resolution of 640×480 pixels.

2. **Binary Thresholding for Object Detection**
   A binary threshold is applied to detect the white ball within the frame. The image is converted into a black-and-white representation where the ball appears as a white region and the background is suppressed.

3. **Morphological Operations**
   Erosion and dilation are applied to the binary mask to remove noise and improve detection accuracy.

4. **Contour Detection**
   Contours are extracted from the processed image. The largest contour is selected as the target object.

5. **Object Localization**
   A bounding rectangle is drawn around the detected object, and the centroid is calculated to determine its position within the frame.

6.3 Grid-Based Motor Control

Instead of continuous angle calculation, the system uses a grid-based control approach. The frame is divided into a 3×3 grid, and each region corresponds to a predefined servo angle.

After detecting the object and computing its centroid, the system determines which grid cell contains the object. Based on this cell, a fixed angle is selected and sent to the Arduino for actuation.

This discrete mapping reduces unnecessary small movements, improves system stability, and minimizes sensitivity to noise.

6.4 Closed-Loop System Operation

The system operates in a continuous feedback loop:

1. Capture image
2. Process image
3. Detect object position
4. Determine grid cell
5. Select corresponding servo angle
6. Send command to Arduino
7. Move servo motor

This ensures that the motor continuously tracks the movement of the object in real time.

7 Results and Performance Evaluation

7.1 System Setup

The physical implementation of the goalkeeper system is shown in Fig. **??** and Fig. **??**. The setup includes the robotic arm, camera module, Raspberry Pi, and Arduino integrated into a single platform.

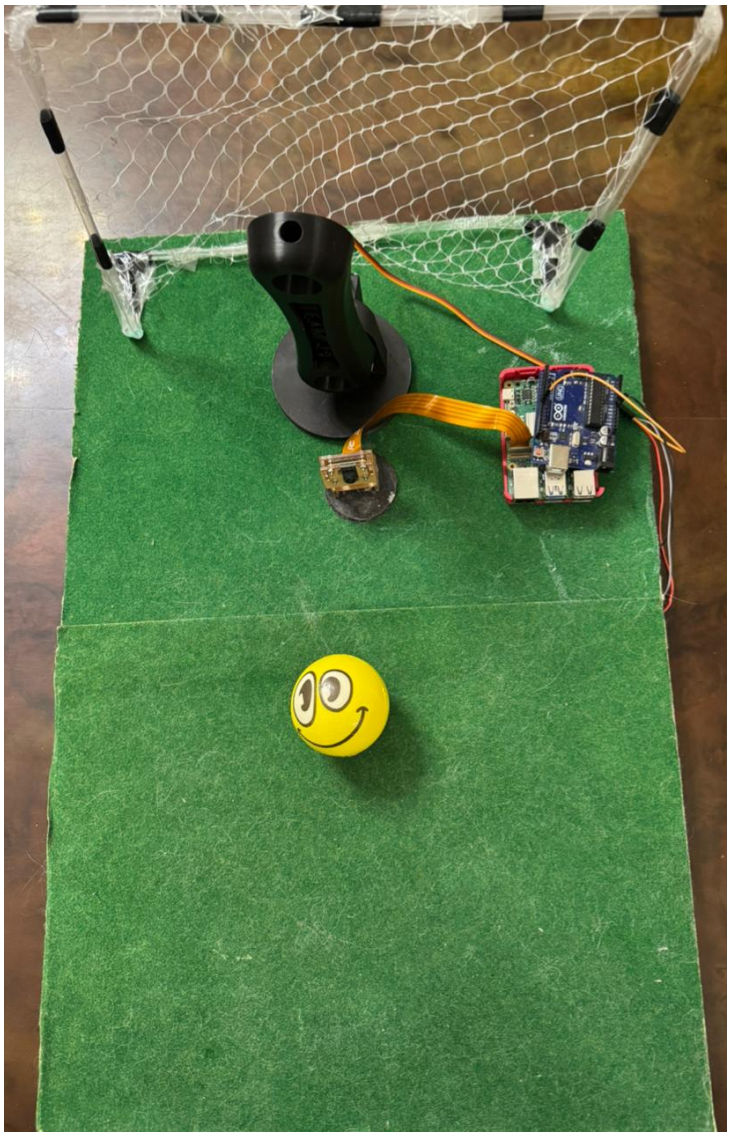

Figure 6: Top view of the experimental setup showing the ball, camera position, and goal structure

Figure 7: Front view of the goalkeeper robotic arm and camera mounting configurations

7.2 Image Processing Results

The image processing pipeline was tested using real-time video input. The system successfully detects the ball using binary segmentation and identifies its position within the grid.

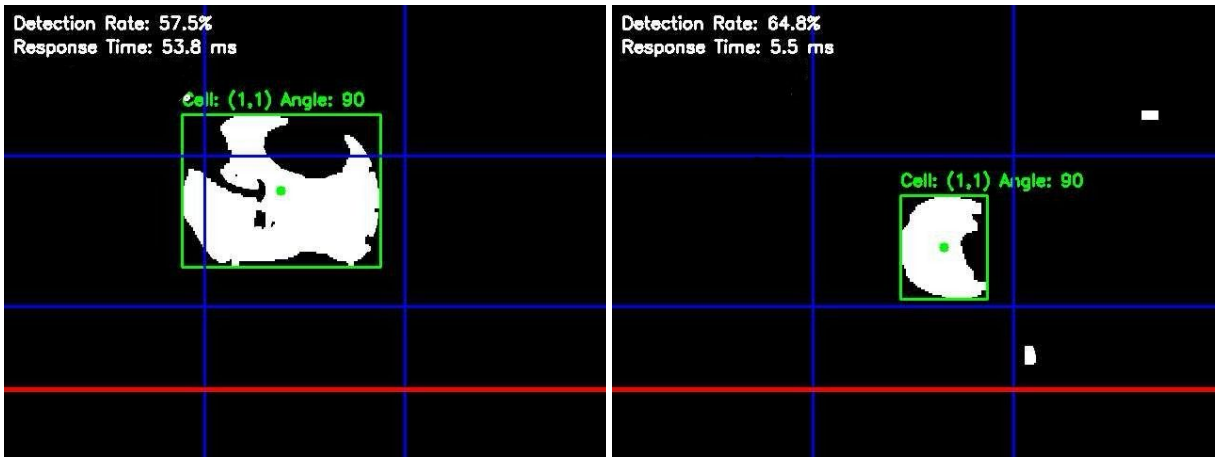

Figure 8: Detected ball with bounding box, centroid, and grid-based position mapping

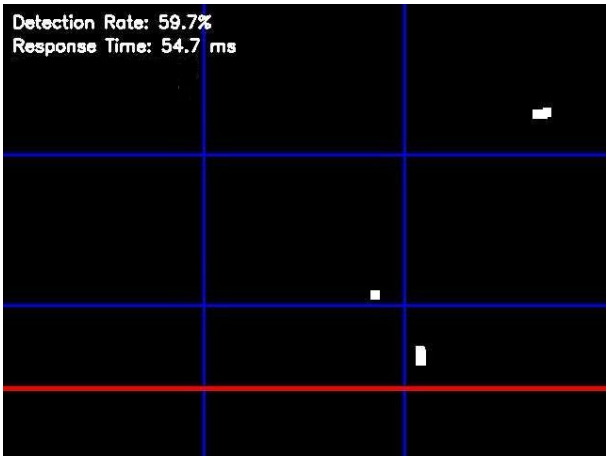

Figure 9: Binary Image showing segmentation of the detected object

The results show that the system is able to isolate the object as a white region in a binary image and compute its centroid. The detected position is mapped to a grid cell, and the corresponding servo angle is displayed.

7.3 Performance Metrics

Metric	**Value**
Detection Accuracy	∼60%
Response Time	∼50 ms
Operating Mode	Real-time

Table 2: Measured performance of the system

7.4 Performance Analysis

The detection accuracy ranged between 57% and 65% depending on noise and lighting conditions. The response time remained within 40–60 ms, indicating that the system operates in real time.

The motor response was consistent with the detected grid position, and the system demonstrated stable behavior due to the use of discrete angle mapping.

7.5 Experimental Conditions

The system was tested under indoor lighting conditions with the ball placed at a distance of approximately 30–50 cm from the camera. The ball was moved manually at moderate speed to simulate real-world conditions.

7.6 Limitations

Despite the successful implementation, several limitations were observed:

- Moderate detection accuracy due to noise in binary segmentation
- Sensitivity to lighting conditions

- Presence of false white regions affecting detection
- No feedback mechanism to verify successful ball blocking
- Reduced accuracy at larger distances

8 Conclusion

This project presented the design and implementation of a real-time goalkeeper robotic arm system using image processing and embedded control. The system successfully detects a white ball, determines its position within the camera frame, and actuates a servo motor accordingly.

An initial brightness-based approach was first explored; however, it showed limitations in terms of sensitivity to lighting conditions and lack of positional accuracy. To address these issues, an improved method based on binary image processing and grid-based control was developed. This modification significantly enhanced system stability and simplified the control strategy.

The system achieved real-time performance with an average response time of approximately 50 ms and a detection accuracy of around 60%. The grid-based approach reduced unnecessary motor movements and improved consistency in actuator response.

Despite these achievements, some limitations remain, including moderate detection accuracy and sensitivity to noise and lighting variations. Additionally, the absence of a feedback mechanism limits the ability to quantitatively evaluate successful ball interception.

Overall, the project demonstrates the feasibility of integrating computer vision techniques with embedded systems for real-time robotic applications. Future improvements may include adaptive thresholding, more robust detection algorithms, and the incorporation of feedback sensors to evaluate system performance more accurately.

References

[1] A. Harikumar, S. K. Saha, and A. R. Sharma, "Colored Ball Position Tracking Method for Goalkeeper Humanoid Robot Soccer," 2020.

[2] D. Santos, P. Ribeiro, and A. Lopes, "Vision-Based Robotic Ball Catching Using Motion Prediction," *Robotics and Autonomous Systems*, vol. 148, 2021.

[3] M. Patel, J. Li, and R. Kumar, "Robotic Arm Interception of Fast-Moving Balls," *IEEE Access*, vol. 10, 2022.

[4] M. A. Khan, S. Rehman, and T. Hussain, "Real-Time Ball Detection Using Computer Vision Techniques," *Sensors*, vol. 23, no. 8, 2023.

[5] R. Patel and K. Sharma, "Ball Detection and Tracking Using Hough Transform and Kalman Filter," *IEEE Access*, 2022.

[6] J. Lee and H. Kim, "Vision-Based Moving Object Tracking Using Kalman Filter for Robotic Applications," *Electronics*, 2021.

[7] S. M. Alim and R. S. Kumar, "Embedded Servo Control for Robotic Arms Using Arduino and PID Controllers," *IEEE Embedded Systems Letters*, 2022.

[8] Y. Zhang, L. Chen, and H. Wang, "Deep Learning-Based Ball Detection for Real-Time Sports Applications," *IEEE Access*, 2023.